O'Regan · Ch 7.4 – 7.5

Advanced Logic

Intuitionistic Logic & Undefined Values

LPF · Parnas · Dijkstra

Formal Methods in Software Engineering | Based on O'Regan (Springer, 2017)

- Classical vs Constructive
- BHK Interpretation
- LPF Logic
- Parnas Tables
- Dijkstra's Guards

Lecture Overview

Ch 7.4

Intuitionistic Logic

Constructive proofs · BHK interpretation · Law of Excluded Middle · Modal translation

Ch 7.5

Undefined Values

The problem of partiality · Three-valued logic approaches

Ch 7.4a

Key Operators

Intuitionistic connectives · Heyting algebras · Propositional rules

Ch 7.5a

LPF, Parnas & Dijkstra

Logic of Partial Functions · Parnas tabular notation · Dijkstra's guarded commands

Intuitionistic Logic

Ch 7.4

> *"A mathematical statement is only true if we can constructively prove it — existence of a proof is more important than abstract truth."*

Classical Logic

- ✗ Bivalent: every proposition is TRUE or FALSE
- ✗ P ∨ ¬P (Law of Excluded Middle) always holds
- ✗ ¬¬P → P (Double Negation Elimination) valid
- ✗ Existence proofs need NOT be constructive
- ✗ Truth is objective & independent of proofs

Intuitionistic Logic

- ✓ A proposition is true only if we have a proof
- ✓ P ∨ ¬P NOT always valid (must prove one side)
- ✓ ¬¬P → P does NOT hold in general
- ✓ Existential proofs must exhibit a witness
- ✓ Truth = provability (constructive foundation)

Historical Background: Brouwer's Intuitionism

1907

L.E.J. Brouwer

PhD thesis introduces intuitionism. Mathematics is a mental activity — not formal symbol manipulation. Only constructively provable objects exist.

1930

Arend Heyting

Formalises intuitionistic logic as a proof system (Heyting calculus). Introduces the Brouwer–Heyting–Kolmogorov (BHK) interpretation of connectives.

1932

Kurt Gödel

Shows intuitionistic logic embeds into classical modal logic S4 via a translation, revealing a deep connection between the two systems.

1945

Kleene/Curry

Curry–Howard correspondence: intuitionistic proofs correspond to programs (propositions-as-types). Foundation of type theory and functional programming.

The BHK Interpretation

Brouwer · Heyting · Kolmogorov

A proof of a compound proposition is defined recursively by the proofs of its components:

$P \wedge Q$ — Conjunction

A pair (p, q) where p is a proof of P and q is a proof of Q

$\neg P$ — Negation

A function that converts any proof p of P into a proof of $\bot$ (falsum/contradiction)

$P \vee Q$ — Disjunction

Either (0, p) with p a proof of P, or (1, q) with q a proof of Q — must choose a side!

$\exists x\ P(x)$ — Existence

A pair (a, p) where a is a specific witness and p is a proof that P(a) holds

$P \rightarrow Q$ — Implication

A function f that converts any proof p of P into a proof f(p) of Q

$\forall x\ P(x)$ — Universal

A function f that, given any element a, produces a proof f(a) that P(a) holds

Key Differences: What Intuitionistic Logic Rejects

Law of Excluded Middle (LEM)

$P \vee \neg P$

Classical: Always a tautology in classical logic — every proposition is true or false with no middle ground.

Intuitionistic: NOT generally valid. To assert $P \vee \neg P$, we must either have a proof of P or a proof of ¬P. Without one, we cannot assert the disjunction.

Example: Let P = 'There exist infinitely many twin primes'. We have no proof of P or ¬P — so LEM fails intuitionistically.

Double Negation Elimination (DNE)

$\neg\neg P \rightarrow P$

Classical: Valid in classical logic: if P cannot be false, then P must be true (proof by contradiction).

Intuitionistic: NOT valid. ¬¬P means 'there is no proof that P leads to contradiction', but this does not give us a constructive proof of P itself.

Example: Many classical existence proofs assume $\neg \exists x\, P(x) \rightarrow \bot$ to conclude $\exists x\, P(x)$, but this gives no witness — rejected.

What IS Valid Intuitionistically

$P \rightarrow \neg\neg P$

Classical: Also holds classically, and equivalent to ¬¬P→P by LEM.

Intuitionistic: VALID: Given proof p of P, define f = λ(g: ¬P). g(p) to construct a proof of ¬¬P. The direction P→¬¬P is always constructive.

Also valid: Modus ponens, ∧-intro/elim, →-intro, ∃-intro with witness, ∀-intro with term.

Semantics & Gödel's Modal Translation

Heyting Algebra Semantics

A Heyting algebra is a bounded lattice (H, ≤, ⊤, ⊥) with a relative pseudo-complement operation → satisfying:

$$a \wedge b \leq c \iff a \leq (b \to c)$$

Truth values are open sets of a topological space — NOT just {0,1}

$\top$ = whole space, $\bot$ = empty set

$a \wedge b$ = intersection, $a \vee b$ = union

$a \to b$ = interior of $(a^c \cup b)$

$\neg a$ = interior of a^c (not just complement!)

This explains why ¬¬a ≠ a in general — the double interior operation does not recover the original open set.

Gödel's Modal Translation (1933)

Gödel showed: Intuitionistic logic (IL) embeds faithfully into classical modal logic S4.

Translation T(φ) prefixes every subformula with the modal operator □ ('necessarily'):

$T(P)$	=	$\Box P$
$T(P \wedge Q)$	=	$T(P) \wedge T(Q)$
$T(P \vee Q)$	=	$T(P) \vee T(Q)$
$T(P \to Q)$	=	$\Box(T(P) \to T(Q))$
$T(\neg P)$	=	$\Box\neg T(P)$

Key result: IL ⊢ φ ⟺ S4 ⊢ T(φ). This established a formal bridge between constructive and classical reasoning.

Section 7.5

Undefined Values

Logic of Partial Functions · Parnas · Dijkstra

The fundamental challenge: what happens when expressions have no defined value?

The Problem of Undefined Values

Ch 7.5

In mathematics and software, many expressions are only partially defined. Classical two-valued logic cannot handle this cleanly — it forces every term to have a truth value, even undefined ones.

Common Sources of Undefinedness

`1 / 0`	Division by zero — no defined result in $\mathbb{R}$
`sqrt(-1)`	Square root of negative in real arithmetic
`head([])`	Head of an empty list — no element exists
`A[n] where n ≥ |A|`	Array access out of bounds
`x (uninitialised)`	Reading an uninitialised program variable
`f(x) partial f`	Applying a partial function outside its domain

Classical Logic's Problem

Principle of Bivalence violated

Classical logic demands every proposition be TRUE or FALSE. But '1/0 = 5' is neither — it is meaningless.

Ex Falso Quodlibet

If $\perp$ (false) appears, everything follows. An undefined expression in a classical proof can make the entire argument collapse.

Partial functions everywhere

Real software deals with partial functions constantly. A formal method that ignores partiality is dangerously incomplete.

Three competing solutions

Ch 7.5 presents three approaches: LPF (Logic of Partial Functions), Parnas tabular notation, and Dijkstra's guarded commands.

LPF: Logic of Partial Functions

Three-valued

LPF extends classical logic with a third truth value U (undefined/unspecified) alongside T (true) and F (false). Developed within the VDM formal method context.

Three-Valued Truth Tables in LPF

P ∧ Q

	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

P ∨ Q

	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

Key LPF Properties & Rules

¬U = U	Negation of undefined is undefined
U ∧ F = F	False dominates in conjunction — even over undefined (Kleene logic)
U ∨ T = T	True dominates in disjunction — even over undefined
U = U is undefined	Equality of undefined expressions is itself undefined — not true!
P ∨ ¬P can be U	LEM fails when P is undefined — P∨¬P need not be T
Definedness predicate ↓	t↓ ('t is defined') — key tool to guard partial expressions
t↓ ∧ P(t) → Q(t)	Precondition pattern: only apply P and Q when t is defined

Parnas: Tabular Notation for Partial Functions

DP

David L. Parnas

Pioneer of software engineering modularisation and tabular specifications. IEEE Fellow.

Parnas's Approach

Parnas proposed handling undefined values through precise tabular specifications. A tabular expression divides the domain into exhaustive, mutually exclusive cases — each cell specifies the function's value in that region. This avoids undefined expressions by making all cases explicit.

Worked Example: sign(x) — the sign function

Condition (domain partition)	Result
x < 0	-1
x = 0	0
x > 0	+1

The table is a completeness proof: the three conditions x<0, x=0, x>0 partition all reals — no undefined case exists.

Tabular Properties

Completeness:
Every input in domain must appear in exactly one row — no case left undefined

Disjointness:
Row conditions are mutually exclusive — no overlapping cases

Verifiability:
Tables can be formally checked for completeness and disjointness using a theorem prover

Readability:
Non-mathematicians can read and review tabular specs — practical for industry

Dijkstra: Guarded Commands & wp-calculus

EW

Edsger W. Dijkstra

ACM Turing Award 1972. Introduced predicate transformers and wp-calculus in 'A Discipline of Programming' (1976).

Dijkstra's Approach to Undefined Values

Dijkstra handles undefinedness by requiring that programs are only executed when all expressions are well-defined. The weakest precondition (wp) calculus computes, for each command S and postcondition Q, the weakest condition under which S is guaranteed to terminate and establish Q — automatically excluding undefined cases.

The Guarded Command Language (GCL)

`skip`	Do nothing (identity)
`abort`	Never terminates (miracle/undefined)
`x := E`	Assignment (only valid when E is defined)
`G → S`	Guarded command: execute S only if guard G is true
`if G₁→S₁ ▯ G₂→S₂ fi`	Selection: choose any branch whose guard is true; abort if none
`do G₁→S₁ ▯ G₂→S₂ od`	Repetition: repeat while some guard is true; terminate when all false

Weakest Precondition Rules

`wp(skip, Q)`	`= Q`
`wp(abort, Q)`	`= false`
`wp(x:=E, Q)`	`= Q[x/E]  (E defined)`
`wp(if Gᵢ→Sᵢ, Q)`	`= (∃i: Gᵢ) ∧ ∀i: Gᵢ→wp(Sᵢ,Q)`
`wp(S;T, Q)`	`= wp(S, wp(T, Q))`

The guard ensures all expressions in S are defined before execution — undefinedness is prevented by construction, not handled after the fact.

Comparison: Three Approaches to Undefinedness

LPF

Logic of Partial Functions

Mechanism:
Third truth value U (undefined) — three-valued logic

LEM:
P ∨ ¬P may be U when P involves undefined terms

Strength:
Logically principled; integrates cleanly into formal specification (VDM)

Weakness:
Reasoning with three values is more complex; tools less mature

Use case:
VDM specifications; reasoning about partial operations on data types

Key author:
Jones (VDM), Barringer, Cheng & Jones

Parnas

Tabular Notation

Mechanism:
Domain partitioned into exhaustive, disjoint cases in a table

LEM:
Avoids issue entirely — all cases are explicitly covered by design

Strength:
Very readable; non-experts can review tables; completeness is checkable

Weakness:
Tables can become large; requires careful domain decomposition upfront

Use case:
Safety-critical systems; requirements tables; interface specifications

Key author:
Parnas (1992–2001), widely used in aerospace and nuclear domains

Dijkstra

Guarded Commands / wp

Mechanism:
Guards prevent execution when operands undefined; wp calculus

LEM:
Avoids issue by design — undefined expressions never reached

Strength:
Prevents undefinedness rather than handling it; clean program logic

Weakness:
Requires programmers to specify guards explicitly; can be verbose

Use case:
Program correctness proofs; refinement calculus; B-method

Key author:
Dijkstra (1976), Morgan (refinement), Back & von Wright

Worked Example: Safe Array Access

Problem: access A[i] where A is an array of length n. This is undefined when $i < 0$ or $i \geq n$.

LPF Approach

$A[i]\downarrow \equiv 0 \leq i < n$

Specification:

pre: $0 \leq i < n$

val: $A[i]\downarrow \wedge result = A[i]$

If i is out of range:

A[i] is undefined (value U)

Any predicate on A[i] has value U

LEM: $A[i]=x \vee A[i]\neq x$ may be U

Parnas Table

Condition	Result
$0 \leq i \wedge i < n$	A[i] (defined)
$i < 0$	ERROR
$i \geq n$	ERROR

Table is complete & disjoint: covers all integer values of i.

No undefined cell exists.

Dijkstra Guard

Guarded access:

```
if 0 ≤ i < n →
  result := A[i]
[] ¬(0 ≤ i < n) →
  abort
fi
```

wp(if ..., result=A[i])

= $(0 \leq i < n)$

Precondition computed!

All three approaches prevent the same bug — but through different philosophies: LPF accommodates undefinedness, Parnas avoids it by case analysis, Dijkstra blocks it with guards.

Key Takeaways

1

Intuitionistic Logic

Truth = provability. Constructive proofs only. LEM and DNE rejected. BHK gives meaning to every connective. Heyting algebras provide semantics.

2

Historical Roots

Brouwer (1907) founded intuitionism. Heyting formalised it (1930). Gödel translated IL into modal logic S4. Curry-Howard: proofs = programs.

3

Undefined Values (LPF)

Three-valued logic: T, F, U. Undefinedness propagates through expressions. LEM may fail. Definedness predicate $t\downarrow$ used to guard partial functions.

4

Parnas Tabular Notation

Exhaustive, disjoint case tables. Completeness and disjointness are verifiable properties. Readable by non-mathematicians. Used in safety-critical industry.

5

Dijkstra / wp-calculus

Guarded commands prevent undefined execution by construction. wp(S,Q) computes the weakest precondition mechanically. Foundation of program refinement.

O'Regan · A Concise Guide to Formal Methods · Springer 2017 · Ch 7.4–7.5